

# PageRank Analysis Using Iterative and Eigenvalue Methods on Directed Web Graphs

LILY NGUYEN<sup>1</sup> AND ANDRE SAE<sup>1</sup>

<sup>1</sup>*Department of Physics, The University of Texas at Austin  
Austin, TX 78712, USA*

## ABSTRACT

In this report, we explore the numerical methods to find the PageRank ranking. The PageRank matrix is a transition matrix that describes the probability of going from one page to another page. And the eigenvector is the ranking weights. The first method we use is the iterative method, which repeatedly applies the PageRank matrix to an equally weighted probability vector to see if it converges. If it converges to particular values that vector is the long term steady state vector describing the page that most people will end up over a long enough time span. Another method we explored was a direct method, where we compute the steady-state eigenvector where the eigenvalue is 1. We also look at the limitation of the ergodic requirement for these methods and suggest a perturbation method known as "random surfer" to resolve that limitation.

*Keywords:* stochastic matrix — hyperlink — directed graph — PageRank — eigenvalue — eigenvector — internet — search engine

## 1. INTRODUCTION

The PageRank algorithm, first introduced by Larry Page and Sergey Brin, was one of the first algorithms to rank webpages based on their link structure rather than solely content [Brin & Page \(1998\)](#). The algorithm models the web as a directed graph and simulates an "agent" who follows hyperlinks with some probability. Essentially, it tracks how often this "agent" would land on a particular page over an extended period, so pages that are visited more frequently are considered more important and thus receive a higher PageRank score [Page et al. \(1999\)](#).

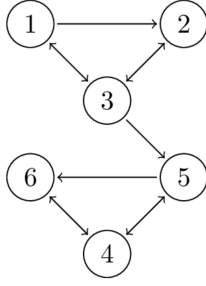
This setup defines a Markov process where each page's importance is proportional to the importance of pages linking to it. The final PageRank vector corresponds to the steady-state distribution of this process and can be found either by iteration or by solving for the dominant eigenvector of the hyperlink matrix  $\mathbf{H}$  [Langville & Meyer \(2009\)](#).

In this project, we first implemented PageRank using the difference equation  $\mathbf{x}_{k+1} = \mathbf{H}\mathbf{x}_k$ , which resulted in the scores converging once the solution reached the maximum tolerance allowed. We then confirmed our result by solving for the eigenvector associated with the largest eigenvalue of the matrix  $\mathbf{H}$ . Our goal is to understand how the algorithm behaves under different net-

work structures and how quickly it converges [Langville & Meyer \(2004\)](#).

## 2. DATA AND OBSERVATIONS

For an  $n \times n$  hyperlink matrix  $\mathbf{H}$  to be stochastic, each column must be a probability vector with non-negative entries that sum to 1. The matrix must also be ergodic which means that cycles do not form and the matrix cannot be reduced to separate independent stochastic matrices. This ensures that probability is conserved at each step of the Markov process and allows the system to converge to a unique steady-state solution. This condition is only satisfied if every page has at least one outgoing link. Otherwise, the corresponding column in  $\mathbf{H}$  would contain all zeroes and violate the stochastic property.



**Figure 1.** Stochastic Matrix Graph

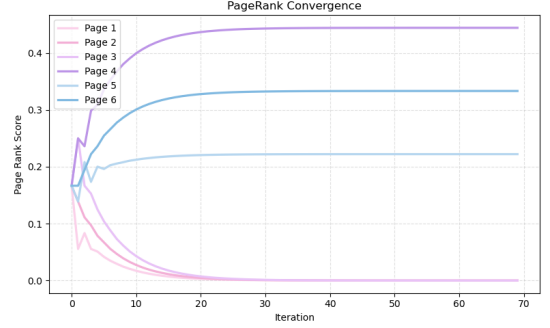
To start, we defined a 6-node directed graph where each page links to one or more pages, as shown in Figure 1. From this, we constructed the hyperlink matrix  $\mathbf{H}$ , where each non-zero entry  $\mathbf{H}_{ij}$  is equal to  $\frac{1}{|P_{ij}|}$ , the reciprocal of the number of outbound links from page  $P_j$ . This matrix represents the probability that an agent on page  $j$  clicks through to page  $i$ :

$$\mathbf{H} = \begin{pmatrix} 0 & 0 & 1/3 & 0 & 0 & 0 \\ 1/2 & 0 & 1/3 & 0 & 0 & 0 \\ 1/2 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1/2 & 1 \\ 0 & 0 & 1/3 & 1/2 & 0 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 & 0 \end{pmatrix} \quad (1)$$

We initialized the PageRank vector  $\mathbf{x}_0$  with equal values and assumed that each page is equally likely to be visited at first:

$$\mathbf{x}_0 = \begin{pmatrix} 1/6 \\ 1/6 \\ 1/6 \\ 1/6 \\ 1/6 \\ 1/6 \end{pmatrix} \quad (2)$$

We then applied the iterative update rule  $\mathbf{x}_{n+1} = \mathbf{H}\mathbf{x}_n$  until the rank vector converged below a given tolerance to simulate an agent moving through the web graph via the links. We visualized the convergence of the PageRank vector by plotting each page's rank score for each iteration, which allowed us to track how the scores changed until they reached a steady state. Our results are shown in Figure 2.



**Figure 2.** PageRank convergence for graph 1.

The convergence of the iterative method is a direct consequence of the underlying theory of stochastic matrices. According to Theorem 4.9, if  $\mathbf{H}$  is an  $n \times n$  stochastic matrix and  $\mathbf{x}_0$  is some initial vector, then the system defined by the difference equation  $\mathbf{x}_{n+1} = \mathbf{H}\mathbf{x}_n$  converges to a unique steady-state vector. This limit corresponds to the eigenvector  $\mathbf{v}_1$  associated with eigenvalue  $\lambda = 1$ , since all the other eigenvalue components vanish as  $k \rightarrow \infty$ :

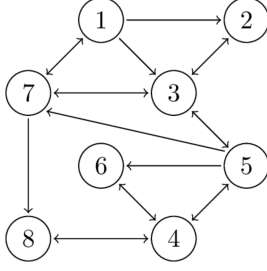
$$\mathbf{x}_{\text{equilibrium}} = \lim_{k \rightarrow \infty} \mathbf{H}^k \mathbf{x}_0 = c_1 \mathbf{v}_1 \quad (3)$$

Theorem 4.9 greatly simplifies the PageRank iterative process. Instead of repeatedly applying  $\mathbf{x}_{n+1} = \mathbf{H}\mathbf{x}_n$ , we can directly compute the dominant eigenvector of  $\mathbf{H}$  corresponding to the eigenvalue  $\lambda = 1$ . This vector represents the steady-state solution and results in the same PageRank distribution as the limit of the iterative method.

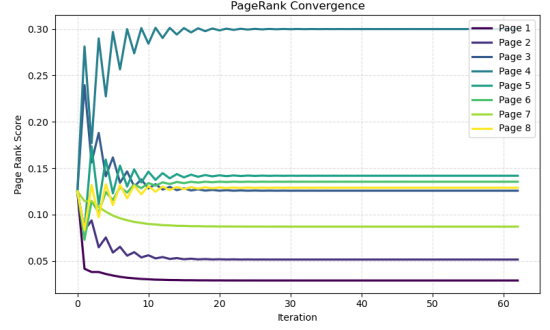
Both the iterative difference equation and the eigenvector method yield the same steady-state PageRank vector:

$$\mathbf{x}_{\text{converged}} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 4/9 \\ 2/9 \\ 1/3 \end{pmatrix} \quad (4)$$

We applied the PageRank algorithm to a second web graph consisting of eight interconnected pages, as shown in Figure 3.



**Figure 3.** example graph 2.



**Figure 4.** PageRank Convergence for graph 2.

<sup>99</sup> The corresponding hyperlink matrix  $\mathbf{H}$  and initial uni-  
<sup>100</sup> form state vector  $\mathbf{H}$  are defined below.

$$\mathbf{H} = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 1/3 & 0 \\ 1/3 & 0 & 1/3 & 0 & 0 & 0 & 0 & 0 \\ 1/3 & 1 & 0 & 0 & 1/4 & 0 & 1/3 & 0 \\ 0 & 0 & 0 & 0 & 1/4 & 1 & 0 & 1 \\ 0 & 0 & 1/3 & 1/3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1/3 & 1/4 & 0 & 0 & 0 \\ 1/3 & 0 & 1/3 & 0 & 1/4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1/3 & 0 & 0 & 1/3 & 0 \end{pmatrix} \quad (5)$$

$$\mathbf{x}_0 = \begin{pmatrix} 1/8 \\ 1/8 \\ 1/8 \\ 1/8 \\ 1/8 \\ 1/8 \\ 1/8 \\ 1/8 \end{pmatrix} \quad (6)$$

<sup>103</sup> Using both the iterative update method and the dom-  
<sup>104</sup> inant eigenvector approach, we computed the steady-  
<sup>105</sup> state PageRank distribution. The convergence behavior  
<sup>106</sup> of the iterative method is shown in Figure 4

<sup>107</sup> Using both the iterative difference equation and the  
<sup>108</sup> dominant eigenvector method, we arrive at the same  
<sup>109</sup> steady-state PageRank vector:

$$\mathbf{x}_{\text{converged}} = \begin{pmatrix} 0.0290 \\ 0.0516 \\ 0.1258 \\ 0.3000 \\ 0.1419 \\ 0.1355 \\ 0.0871 \\ 0.1290 \end{pmatrix} \quad (7)$$

### 3. RESULTS

This distribution for the first web graph reflects the long-term behavior of an agent navigating the web graph. Based on the values, we rank the pages in order of importance as follows:

Page 4 > Page 6 > Page 5 > Page 1 = Page 2 = Page 3

<sup>112</sup> We know that after a long enough time span,  $\frac{4}{9}$  of a  
<sup>113</sup> group of people will end up in Page 4,  $\frac{2}{9}$  of a group of  
<sup>114</sup> people will end up in page 5, and  $\frac{1}{3}$  of a group people  
<sup>115</sup> will end up in Page 1. No one will end up on Page 1, 2,  
<sup>116</sup> and 3.

From this second web graph result, we rank the pages in order of importance:

Page 4 > Page 5 > Page 6 > Page 8 > Page 3

> Page 7 > Page 2 > Page 1

<sup>117</sup> Page 4 is the most dominant, whereas Page 1 has the  
<sup>118</sup> lowest influence in the network. In order word most peo-  
<sup>119</sup> ple will end up on page 4 and the least amount of people  
<sup>120</sup> will end up on page 1 given a long enough timespan.

#### 4. SUMMARY AND CONCLUSION

To summarize, we implemented two numerical approaches to compute the PageRank vector. The first was an iterative method that started from a uniform initial distribution  $\mathbf{x}_0$  and repeatedly applied the update equation  $\mathbf{x}_{n+1} = \mathbf{H}\mathbf{x}_n$  until the difference between successive iterations fell below a convergence threshold. The second approach used Theorem 4.9, which states that the steady-state distribution is the dominant eigenvector of the stochastic matrix  $\mathbf{H}$ . Both methods gave identical results, which confirms the accuracy of our implementation to describe long term internet behavior. We performed all computations in Python using NumPy and visualized convergence behavior with matplotlib.

The biggest limitation is that each iteration is assumed to be independent, which means that the current page and previous that the user is on does not influence the next page the user will explore. This condition might hold when the internet was more decentralized. Given the rise of modern recommendation algorithm on platforms that tries to maximize user retention, this assumption doesn't hold true.

Another limitation is that the PageRank matrix  $\mathbf{H}$  must be ergodic, such as the one that we've shown in the article. In order to solve this issue a perturbation can be made to the original matrix to make all matrices ergodic. This is done by introducing a dampening factor  $\alpha$  which represent the dampening factor,  $\mathbf{J}$  is a ones matrix, and  $n$  is the number of pages.

$$\hat{H} = (1 - \alpha)H + \frac{\alpha}{n}J \quad (8)$$

This perturbation represents the chance that the user has of jumping to another page randomly. It has a nice property of preventing loops from forming, while also summing the probabilities for one page to jump to another page up to one which defines a stochastic matrix.

#### 4.1. Acknowledgements

We thank Professor Mitra for his guidance and instruction throughout the course, as well as the teaching assistants for their helpful feedback. We also acknowledge the Department of Physics for providing an excellent academic foundation to support this work. This work was completed as part of the requirements for C S 323E: Elements of Scientific Computing at the University of Texas at Austin.

#### REFERENCES

- Brin, S., & Page, L. 1998, Computer Networks, 30, 107.  
<https://snap.stanford.edu/class/cs224w-readings/Brin98Anatomy.pdf>
- Langville, A. N., & Meyer, C. D. 2004, Internet mathematics, 1, 335
- . 2009, Google's pagerank and beyond: The science of search engine rankings., 1st edn. (Princeton, N.J: Princeton University Press)
- Page, L., Brin, S., Motwani, R., & Winograd, T. 1999, The PageRank Citation Ranking: Bringing Order to the Web., Technical Report 1999-66, Stanford InfoLab.  
<http://ilpubs.stanford.edu:8090/422/>